



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN GENERAL DE PREGRADO

INGENIERÍA INFORMÁTICA

ASIGNATURA: INGENIERÍA DEL SOFTWARE II

SECCIÓN 1

“Gestión de Riesgos y Deuda Técnica”

DOCENTE:

Félix Márquez

ALUMNOS:

Luis Millán **V - 30.040.213**

Endrys Flores **V - 30.451.556**

Robert Castro **V - 30.994.049**

Jesus Báez **V - 26.753.871**

Ciudad Guayana, Septiembre del 2026.

ÍNDICE

INTRODUCCIÓN	3
Identificación y Evaluación de Riesgos Críticos (MediConnect)	4
Matriz de Priorización de Riesgos	6
Planes de Contingencia y Mitigación Financiera/Temporal	6
Diseño del Registro de Riesgos Híbrido (Risk Register)	8
Mecanismo de Gobernanza Ágil (Integración en Ceremonias Scrum)	10
Transformación de Riesgo a Product Backlog (Ejemplo Práctico)	11
Análisis Comparativo: Sesión Tradicional vs. Risk Storming Multifuncional	13
¿Por qué Risk Storming es superior para arquitecturas de software y ciberseguridad?	14
La Deuda Técnica como Riesgo de Negocio: TDR (SQALE) y Mecanismos SLA/WAF	15
CONCLUSIONES	17
REFERENCIAS	20

INTRODUCCIÓN

La administración de riesgos en la ingeniería de software actual ha superado el método clásico de matrices estáticas diseñadas al comienzo del proyecto. Actualmente, se presenta como una disciplina dinámica, iterativa y presente en todo el ciclo de vida del producto, enfocándose no en eliminar la incertidumbre —una condición inherente a los sistemas complejos— sino en fomentar la resiliencia organizacional y en tomar decisiones informadas que salvaguarden el valor del producto y la sostenibilidad del equipo.

Este informe registra la implementación de los conceptos abordados en la Unidad IV: Gestión de Riesgos y Deuda Técnica, utilizando como caso de estudio el proyecto MediConnect, una plataforma de telemedicina que combina videoconsultas WebRTC, registros médicos electrónicos, sistemas de pago y una API de Inteligencia Artificial externa para el diagnóstico asistido. La selección de este ámbito no es casual: la telemedicina aglutina, en un único ecosistema, los riesgos contemporáneos más significativos de la industria actual —arquitecturas distribuidas, dependencia de proveedores en la nube, manejo de datos de salud sensibles (PHI/HIPAA), integración de modelos de IA no deterministas y una ruta crítica sin ningún margen de manobra—, lo que lo hace un contexto perfecto para aplicar técnicas de identificación, análisis, priorización y reducción de riesgos.

El texto se organiza en cinco partes. Se inicia con el Risk Storming simulado, en el que se identifican y detallan tres riesgos fundamentales: (R1) la interrupción de sesiones WebRTC en videoconsultas, de tipo técnico y arquitectónico; (R2) la vulneración de la privacidad y la

exposición de datos de salud, de naturaleza legal y de normativa; y (R3) el punto crítico en la ruta esencial debido al acoplamiento en el backend central, de carácter de gestión y de desarrollo.

Identificación y Evaluación de Riesgos Críticos (MediConnect)

- **R1: Caída de sesiones WebRTC durante videoconsultas (Técnico)**
 - **Categoría:** Arquitectura y Performance (Módulo 4.3 Telemedicina).
 - **Probabilidad / Impacto:** Alta / Muy alto.
 - **Análisis Cualitativo:** Interrupción del flujo clínico en tiempo real, insatisfacción del médico/paciente y pérdida de reputación de la plataforma.
 - **Análisis Cuantitativo:** Riesgo de retraso de 5 días en el bloque de desarrollo (Sprints 4.1-4.4). Pide consumo adicional de infraestructura redundante STUN/TURN (+15% sobre presupuesto de nube).
 - **Mitigación / Contingencia:** Implementar fallback automático a llamada de voz/chat en caso de degradación de red y servidores TURN redundantes en distintas regiones.
- **R2: Brecha de privacidad y filtración de Datos de Salud (Seguridad / Cumplimiento)**
 - **Categoría:** Seguridad y Normativa (Módulos 2.2, 4.4 Historias Médicas y Pagos).
 - **Probabilidad / Impacto:** Baja / Crítico.
 - **Análisis Cualitativo:** Sanciones legales graves por incumplimiento de normativas de protección de datos médicos y pérdida total de confianza institucional.

- **Análisis Cuantitativo:** Costo potencial de penalizaciones legales e impacto directo sobre la fase 5.0 (QA, Testing & UAT), exigiendo pruebas de penetración adicionales que consumirían 4 días de la holgura del proyecto.
- **Mitigación / Contingencia:** Encriptación End-to-End en videollamadas, cifrado AES-256 en base de datos para historias clínicas y tokenización estricta de pasarelas de pago.
- **R3: Cuello de botella en la Ruta Crítica por acoplamiento en Backend Core (Gestión / Programación)**
 - **Categoría:** Cronograma y Dependencias (Tarea 4.1 Core Backend).
 - **Probabilidad / Impacto:** Media / Alto.
 - **Análisis Cualitativo:** Dado que la Ruta Crítica es de 80 días con Holgura Total de 0 Días, cualquier retraso en el Core Backend (10 días) desplaza en cadena las fases 4.2-4.4, QA (5.0) y Go-Live (6.0).
 - **Análisis Cuantitativo:** Un retraso de 3 días en la tarea 4.1 desplaza la fecha de culminación del 25/12/2026 al 30/12/2026.
 - **Mitigación / Contingencia:** Aplicar Capacidad Reservada (15% a 20%) durante los Sprints, diseño desacoplado mediante APIs RESTful y revisión continua del Throughput mediante métricas de flujo.

Matriz de Priorización de Riesgos

ID	Riesgo	Probabilidad	Impacto	Tarea Afectada (EDT)	Estrategia de Respuesta
R1	Fallo WebRTC en telemedicina	Alta	Muy Alto	4.3 Telemedicina	Redundancia STUN/TURN + Fallback a voz
R2	Vulneración de Historias Clínicas	Baja	Crítico	4.4 Historias & Pagos / 5.0 QA	Cifrado AES-256 y auditoría Pentest
R3	Desviación en Core Backend (Ruta Crítica)	Media	Alto	4.1 Core Backend	Desarrollo paralelo de stubs de API y buffer

Planes de Contingencia y Mitigación Financiera/Temporal

- **Presupuesto de Contingencia:** Reservar el 15% del esfuerzo asignado a los Sprints 4.1 a 4.4 para resolver deuda técnica preventiva y correcciones de arquitectura antes de entrar a la fase 5.0 (QA & UAT).
- **Control de Cambios (Scope Creep):** Si se solicitan nuevos requisitos durante los Sprints 4.2-4.4, la inclusión debe aprobarse bajo una permutación de alcance en el backlog (intercambiar historias de usuario de igual tamaño en Story Points), evitando así alterar la fecha límite del 25/12/2026.

Monitoreo de Indicadores de Riesgo (KRI)

- **Cycle Time vs. Target:** Si el tiempo de ciclo por Historia de Usuario supera los 4 días en la etapa 4.1-4.4, se activa alerta roja de desviación de cronograma.
- **Code Churn / Fricción:** Monitoreo semanal de modificaciones masivas de código en el módulo de pagos e historias clínicas para prevenir la acumulación de deuda técnica.

Diseño del Registro de Riesgos Híbrido (Risk Register)

El Registro de Riesgos Híbrido de una sola página permite conciliar la trazabilidad exigida por auditores externos con la visibilidad continua requerida por el equipo de desarrollo.

ID	Descripción del Riesgo	Prob. (1-5)	Imp. (1-5)	Risk Score (P×I)	Dueño (Risk Owner)	Estado	Artefacto / Vínculo Ágil
RSK-MED-01	Cambio inesperado en términos de servicio, límites de cuotas o incrementos de tarifas en la API de IA de diagnóstico.	4	4	16 (Crítico)	Tech Lead / Arquitecto	En Mitigación	PB-EN-104 (Enabler Story en Sprint 12)
RSK-MED-02	Filtración de datos médicos protegidos (PHI/HIPAA) por configuraciones deficientes en la red de la nube (Modelo de Responsabilidad	2	5	10 (Alto)	CISO / SecOps Lead	Abierto	SEC-SPIKE-03 (Investigación Zero Trust)

ID	Descripción del Riesgo	Prob . (1-5)	Imp. (1-5)	Risk Score (P×I)	Dueño (Risk Owner)	Estado	Artefact o / Vínculo Ágil
	Compartida).						
RSK-M ED-03	Acumulación excesiva de Deuda Técnica en la capa de integración con historias clínicas electrónicas (TDR > 15%).	3	3	9 (Medio)	Senior Backend Dev	Monitoreado	PB-DEBT-12 (Refactorización Continua en Backlog)
RSK-M ED-04	Latencia y degradación de servicio en la sincronización de recetas digitales por falla del proveedor de nube único (Vendor Lock-in).	2	4	8 (Medio)	DevOps Lead	Aceptado con Contingencia	CR-RES-02 (Reserva de Contingencia Cloud)

“Nota de Auditoría: El registro se sincroniza automáticamente mediante pipelines de CI/CD e integración con la herramienta de gestión ágil (Jira/Azure DevOps), garantizando inmutabilidad, logs de cambios y exportación en formatos estandarizados para auditorías externas.”

Mecanismo de Gobernanza Ágil (Integración en Ceremonias Scrum)

Para evitar burocracia desmedida y mantener el Registro de Riesgos vivo, la revisión y evaluación de riesgos se sincroniza de forma nativa en cada una de las ceremonias del framework Scrum:

- **Sprint Planning:**

- **Acción:** Al planificar la capacidad del Sprint, el Product Owner y el equipo técnico evalúan el *Risk Score* de los ítems del Product Backlog seleccionados.
- **Gobernanza:** Si un riesgo supera el umbral crítico (≥ 12), se exige la inclusión obligatoria de al menos un *Enabler* o *Spike* de mitigación dentro del Sprint Backlog actual.

- **Daily Scrum (15 min):**

- **Acción:** El equipo identifica impedimentos operacionales o riesgos emergentes durante la ejecución diaria.
- **Gobernanza:** Cualquier bloqueo técnico con potencial de impacto en seguridad, arquitectura o costos se escala inmediatamente al Risk Owner sin esperar a reuniones semanales de seguimiento.

- **Sprint Review:**

- **Acción:** Se presenta a los *stakeholders* y auditores el avance tangible de las tareas de mitigación técnicas implementadas en el incremento del producto.
- **Gobernanza:** Reclasificación del estado del riesgo (ej. de *En Mitigación* a *Monitoreado*) con base en la evidencia funcional presentada.

- **Sprint Retrospective:**

- **Acción:** El equipo realiza una sesión breve de *Risk Storming* ligero al revisar el proceso de ingeniería.
- **Gobernanza:** Se reevalúa la probabilidad e impacto de riesgos existentes y se ingresan nuevos riesgos identificados al Registro Híbrido.

Transformación de Riesgo a Product Backlog (Ejemplo Práctico)

A continuación se presenta la transformación operacional del riesgo crítico

RSK-MED-01 ("*Cambio en términos/costos de la API de IA*") en una Enabler User Story priorizada dentro del Product Backlog.

- **ID:** PB-EN-104
- **TÍTULO:** Implementación de Capa de Abstracción de IA (Adapter Pattern) y Sistema de Caché de Diagnóstico TIPO DE ÍTEM: Enabler Story (Arquitectura / Infraestructura)
- **PRIORIDAD:** Alta (Derivada de Risk Score = 16)
- **ESTIMACIÓN:** 8 Puntos de Historia
- **DESCRIPCIÓN:** Como Arquitecto de Software de MediConnect, Quiero desacoplar el módulo de diagnóstico médico de la API del proveedor externo actual mediante una interfaz genérica y una capa de almacenamiento en caché, Para reducir la dependencia directa de un único proveedor, amortiguar incrementos de tarifas y permitir la conmutación a un modelo alternativo sin interrumpir el servicio.

CRITERIOS DE ACEPTACIÓN:

1. **Desacoplamiento Arquitectónico:** La aplicación debe interactuar únicamente con la interfaz 'IAssistiveDiagnosticService'. La lógica específica del proveedor actual residirá en un módulo aislado (Plugin/Adapter Pattern).
2. **Optimización de Costos y Consultas:** Implementar un almacenamiento en caché con Redis para respuestas diagnósticas repetitivas, reduciendo las llamadas a la API externa en al menos un 30%.
3. **Conmutación en Tiempo de Ejecución:** El sistema debe permitir cambiar el proveedor de IA subyacente modificando variables de entorno de configuración, sin requerir cambios o redespílegues de código fuente.
4. **Verificación de Resiliencia:** La suite de pruebas de integración continua (CI) debe validar de forma automatizada la conmutación al adaptador de respaldo en menos de 5 minutos ante simulaciones de falla o timeout.

Análisis Comparativo: Sesión Tradicional vs. Risk Storming Multifuncional

La identificación de riesgos en un proyecto de software puede abordarse desde dos enfoques con supuestos epistemológicos distintos: una sesión tradicional, liderada exclusivamente por el Gerente de Proyectos apoyado en una lista de verificación (checklist) construida a partir de lecciones aprendidas (Sommerville, 2016; PMI, 2021); y una sesión de Risk Storming, una dinámica colaborativa y visual en la que el equipo multifuncional completo analiza en conjunto un diagrama de arquitectura o de flujo de usuario para identificar amenazas.

Criterio	Sesión tradicional (Gerente + checklist)	Risk Storming (equipo multifuncional)
Fuente del conocimiento	Una sola persona (el Gerente), basada en lecciones aprendidas de proyectos anteriores.	Todo el equipo (desarrolladores, QA, Product Owner), cada uno con visibilidad de una capa distinta del sistema.
Cobertura de riesgos técnicos/arquitectónicos	Limitada: la checklist detecta riesgos genéricos y ya conocidos, no vulnerabilidades específicas de la arquitectura actual.	Alta: se analiza un diagrama real de arquitectura o flujo de usuario, exponiendo dependencias, integraciones y puntos frágiles concretos.
Detección de riesgos de ciberseguridad	Superficial: rara vez identifica vectores de ataque en la cadena de suministro o configuraciones específicas.	Superior: desarrolladores y QA identifican vulnerabilidades reales (dependencias de terceros, APIs mal configuradas) a partir de su experiencia directa con el código.
Sesgos	Sesgo de autoridad y de disponibilidad: se listan los riesgos que el Gerente recuerda o que ya ocurrieron antes.	Se democratiza la identificación; se reducen los puntos ciegos de una sola perspectiva jerárquica.

Apropiación del equipo	Baja: el equipo recibe una lista ya elaborada, sin haber participado en su construcción.	Alta: al identificar los riesgos colaborativamente, el equipo entiende y se compromete con su propia mitigación.
-------------------------------	--	--

¿Por qué Risk Storming es superior para arquitecturas de software y ciberseguridad?

La superioridad del Risk Storming no es una preferencia metodológica, sino una consecuencia directa de dónde reside el conocimiento técnico en un equipo de desarrollo:

- **Visibilidad técnica distribuida:** un Gerente de Proyectos, por diseño de su rol, rara vez conoce al detalle las dependencias de terceros, las configuraciones de IAM o los puntos de integración frágiles de la arquitectura. Esa información reside en los desarrolladores y en QA.
- **Los riesgos de ciberseguridad modernos son específicos del contexto:** incidentes como Log4j o SolarWinds (OWASP, 2021) no se originaron en fallas obvias detectables por una checklist genérica, sino en vulnerabilidades heredadas de dependencias de terceros — visibles únicamente para quien examina el diagrama real de componentes.
- **Reducción de sesgos jerárquicos:** una checklist liderada por una sola persona está sujeta al sesgo de disponibilidad (solo lista lo que ya ha visto fallar antes). El Risk Storming democratiza la detección al exponer el mismo diagrama a múltiples perspectivas simultáneamente.
- **Apropiación y mitigación posterior:** al identificar los riesgos de forma colaborativa, el equipo que luego deberá mitigarlos (convirtiéndolos en Spikes o Historias Habilitadoras en el Product Backlog, Cohn, 2005) ya comprender su causa raíz, acelerando la respuesta.

Una sesión tradicional es adecuada para riesgos de gestión conocidos (cronograma, presupuesto, alcance), pero para riesgos técnicos y de ciberseguridad, el Risk Storming ofrece una cobertura de detección estructuralmente superior.

La Deuda Técnica como Riesgo de Negocio: TDR (SQALE) y Mecanismos SLA/WAF

1. Naturaleza y medición de la deuda técnica

La deuda técnica, concepto acuñado por Ward Cunningham (1992), describe la decisión —consciente o no— de elegir una solución técnica rápida pero subóptima en lugar de una arquitectura bien diseñada. Kruchten et al. (2012) enfatizan que esta deuda genera intereses acumulativos: mayores costos de mantenimiento, menor velocidad de desarrollo futura y mayor fragilidad del sistema. Es importante destacar que no toda deuda es perjudicial: existe deuda deliberada y prudente (una decisión estratégica consciente de acelerar el time-to-market) frente a la deuda inadvertida e imprudente, que es la que genuinamente pone en riesgo la sostenibilidad del producto.

Para operacionalizar este concepto se utiliza el TDR (Technical Debt Ratio) del método SQALE (Software Quality Assessment based on Lifecycle Expectations), que compara el costo de reparar el código frente al costo total de desarrollarlo desde cero:

$$\textbf{TDR} = (\textbf{Costo de remediación} / \textbf{Costo de desarrollo}) \times 100\%$$

Un TDR alto es una señal cuantificable de que se requiere inversión urgente en refactorización —tan medible como cualquier otro KPI financiero—, típicamente calculado con herramientas de análisis estático como SonarQube (IEEE Software, 2020).

2. SLA y WAF como mecanismos de mitigación y transferencia

Frente a los riesgos de infraestructura y ciberseguridad asociados a la deuda técnica y a la dependencia de proveedores externos, se utilizan dos mecanismos de respuesta de naturaleza distinta:

- **SLA (Service Level Agreement)** — mecanismo de transferencia: es un contrato formal con el proveedor de tecnología (AWS, Azure, un SaaS) que define y mide cuantitativamente parámetros mínimos de calidad y disponibilidad (p. ej., uptime del 99.99% anual). Ante un incumplimiento, el SLA obliga al proveedor a compensaciones financieras, desplazando la responsabilidad operativa hacia un tercero capaz de asumirla eficientemente.
- **WAF (Web Application Firewall)** — mecanismo de mitigación técnica: reduce directamente la probabilidad y el impacto de un ataque (p. ej., DDoS o inyección a nivel de aplicación) antes de que llegue a afectar el servicio. Proveedores como Cloudflare —que ha mitigado ataques DDoS de escala récord en tiempos de reacción menores a 3 segundos (Cloudflare, 2025)— o AWS Shield (Amazon Web Services, s.f.) ofrecen esta protección como servicio gestionado, integrado de forma nativa en la infraestructura cloud.

El SLA no reduce la probabilidad de que ocurra una falla, transfiere su costo financiero; el WAF sí reduce directamente la probabilidad de que un ataque tenga éxito. En un proyecto con infraestructura serverless y dependencias de terceros (como una fintech que integra una API de IA externa), ambos mecanismos son complementarios: el SLA protege financieramente ante incumplimientos del proveedor, y el WAF protege técnicamente ante amenazas activas.

CONCLUSIONES

La creación de este informe permitió implementar, en un caso específico como MediConnect, los principios básicos de la gestión de riesgos contemporánea presentados en la Unidad IV. Se resumen las conclusiones más importantes a continuación.

Se verificó que la gestión de riesgos en ingeniería de software resulta efectiva al considerarse un proceso continuo en lugar de un producto final estático. El Risk Storming aplicado al proyecto de telemedicina facilitó la identificación de amenazas que una revisión convencional, dirigida solo por el gerente de proyectos con una lista de verificación, difícilmente habría detectado con igual profundidad: la vulnerabilidad de las sesiones WebRTC ante degradación de red, el riesgo legal por filtración de datos médicos y el acoplamiento del backend core como potenciador de retrasos en una ruta crítica sin holgura. Esto valida que la variedad de opiniones del equipo interdisciplinario —desarrolladores, QA, DevOps, seguridad y Product Owner— mejora considerablemente la identificación de riesgos técnicos y cibernéticos.

En segundo lugar, la diferenciación entre análisis cualitativo y cuantitativo fue operativamente beneficiosa. El primero facilitó la priorización rápida de los riesgos según su probabilidad e impacto (R1 con Alta/Muy Alto, R2 con Baja/Crítico, R3 con Media/Alto), mientras que el segundo cuantificó consecuencias específicas: un retraso de tres días en la tarea

4.1 del Core Backend mueve la fecha de finalización del 25/12/2026 al 30/12/2026, y un incidente de filtración de datos consumiría cuatro días de margen en pruebas de penetración adicionales. Esta doble estrategia posibilita asignar recursos de mitigación de manera proporcional y fundamentada.

Tercero, el Registro de Riesgos Híbrido resultó ser una herramienta efectiva para unir dos mundos que parecen estar en conflicto: la necesidad de trazabilidad y documentación en las auditorías externas, y la rapidez y visibilidad constante que demanda un equipo Scrum. La inclusión de campos como Propietario del Riesgo, Estado y Vínculo Ágil (PB-EN-104, SEC-SPIKE-03, PB-DEBT-12, CR-RES-02) asegura que cada riesgo cuente con un responsable definido y una acción específica en el backlog, evitando que se transforme en un documento inactivo.

Cuarto, la incorporación de la evaluación de riesgos en las ceremonias Scrum —Planificación del Sprint, Scrum Diario, Revisión del Sprint y Retrospectiva del Sprint— evidenció que se puede mantener el registro activo sin incrementar la burocracia. La norma de que cualquier riesgo con un Risk Score mayor al límite crítico requiera la inclusión forzada de al menos un Enabler o Spike en el Sprint Backlog vigente asegura que la mitigación se convierta en trabajo tangible y cuantificable, en lugar de simples intenciones.

Quinto, el caso de transformación del riesgo RSK-MED-01 en la Enabler User Story PB-EN-104 mostró el valor práctico de transformar una amenaza estratégica —el cambio unilateral de términos y costos de la API de IA— en una solución arquitectónica específica: el patrón Adapter, una capa de caché con Redis y el cambio en tiempo de ejecución mediante variables de entorno. Este ejercicio muestra que la deuda técnica y los riesgos de dependencia de

terceros pueden manejarse con el mismo rigor que una característica de negocio, utilizando criterios de aceptación medibles y estimaciones en puntos de historia.

Finalmente, el grupo determina que la gestión de riesgos no es un objetivo en sí, sino una herramienta para resguardar el valor del producto y la sostenibilidad del equipo. En un escenario donde la telemedicina gestiona información delicada, depende de infraestructura en la nube y se fundamenta en modelos de IA no predictivos, la habilidad para prever, clasificar y reducir riesgos se transforma en una habilidad estratégica. La idea fundamental de esta unidad es que la incertidumbre no se erradica: se maneja con claridad, rigor y una cultura en la que los riesgos se abordan de manera abierta en cada iteración. Este informe representa, en este contexto, una prueba concreta de que es factible transformar la teoría en acción en la administración de riesgos y deuda técnica dentro de un enfoque ágil.

REFERENCIAS

PMI (Project Management Institute). (2021). A Guide to the Project Management Body of Knowledge (PMBOK Guide) – Seventh Edition. PMI.

ISO. (2018). ISO 31000:2018 Risk Management — Guidelines. International Organization for Standardization.

Gobernanza Scrum y Riesgos Híbridos: *(Schwaber & Sutherland, 2020; PMI, 2021)*

Cálculo de Ruta Crítica y Holgura: *(Kerzner, 2017)*

Mitigación WebRTC y Servidores TURN: *(Avanwy, 2020; Rescorla, 2018)*

Protección de Datos PHI e HIPAA: *(Department of Health and Human Services, 2003; Pressman & Maxim, 2020)*